

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN – ĐIỆN TỬ

BÁO CÁO CUỐI KỲ DỰ ÁN MÔN HỌC
KIẾN TRÚC MÁY TÍNH

ĐỀ TÀI:

MÔ PHỎNG CPU 8-CORE VỚI
KIẾN TRÚC TẬP LỆNH RISC-V
(KIẾN TRÚC PIPELINE 5 TẦNG)

Nhóm sinh viên thực hiện:

Hoàng Anh Quân – 202418969

Trương Tùng Dương – 202418885

Nguyễn Minh Hoàng – 202418904

Bùi Thị Mai Linh – 20227240

Giảng viên hướng dẫn:

Hội Đồng Bộ Môn Kiến Trúc Máy Tính

Hà Nội, Ngày 18 tháng 6 năm 2026

Tóm Tắt Đóng Góp Của Các Thành Viên

Báo cáo cuối kỳ này ghi nhận kết quả nghiên cứu và thiết kế của cả nhóm. Sự phân chia công việc được thống nhất dựa trên năng lực và tinh thần tự nguyện của từng thành viên, cụ thể như sau:

STT	Thành viên	Nội dung công việc đảm nhận	Đóng góp
1	Hoàng Anh Quân	- Thiết kế kiến trúc lõi đơn pipeline 5 tầng (<code>risc_core.v</code>). - Thiết kế bộ điều khiển Bus (<code>bus_ctrl.v</code>) và bộ phân xử Arbiter (<code>arbiter.v</code>). - Tích hợp toàn hệ thống 8 lõi (<code>top_8core.v</code>).	30%
2	Trương Tùng Dương	- Xây dựng kịch bản kiểm thử tích hợp hệ thống (<code>system_tb.v</code>) và đơn nhân (<code>core_tb.v</code>). - Cấu hình môi trường biên dịch tự động, chạy mô phỏng và phân tích dạng sóng GTKWave.	25%
3	Nguyễn Minh Hoàng	- Thiết kế khối ALU số học/logic (<code>alu.v</code>). - Thiết kế khối Tập thanh ghi (<code>reg_file.v</code>) có cơ chế writeback bypass. - Viết báo cáo kỹ thuật.	23%
4	Bùi Thị Mai Linh	- Thiết kế khối giải mã chỉ thị (<code>control_unit.v</code>). - Xây dựng bộ nhớ dữ liệu shared RAM (<code>data_mem.v</code>) và các ROM chứa lệnh (<code>instr_mem.v</code>).	22%

Bảng 1: Bảng phân công nhiệm vụ và tỷ lệ đóng góp của thành viên

Mục lục

Tóm Tắt Đóng Góp Của Các Thành Viên	1
1 Giới Thiệu Đề Tài	4
1.1 Bối cảnh nghiên cứu	4
1.2 Chủ đề của dự án	4
1.3 Mục tiêu thiết kế	4
2 Phương Pháp Luận & Kiến Trúc Hệ Thống	5
2.1 Phương pháp luận thiết kế hệ thống	5
2.1.1 Quy trình thiết kế Top-Down kết hợp Bottom-Up	5
2.1.2 Mô hình hóa phần cứng ở mức RTL (RTL Modeling)	5
2.1.3 Quy trình mô phỏng và kiểm chứng dựa trên công cụ mã nguồn mở	5
2.1.4 Mô hình hóa chia sẻ tài nguyên và cơ chế đồng bộ	6
2.2 Kiến trúc tập lệnh RISC-V (ISA) áp dụng trong dự án	6
2.2.1 Tổng quan về tập chỉ thị RV32I	6
2.2.2 Tập thanh ghi đa năng (Register File)	6
2.2.3 Các kiểu định dạng chỉ thị lệnh cơ bản	7
2.2.4 Đặc tả định dạng mã hóa và trường bit của các lệnh thực hiện	7
2.2.5 Các bảng tổng hợp đặc tả mã hóa chỉ thị lệnh	9
2.3 Sơ đồ cấu trúc tổng thể hệ thống 8-Core	13
2.4 Thiết kế đường truyền dữ liệu (Datapath) đơn nhân	13
2.4.1 Tầng Nạp chỉ thị (Instruction Fetch - IF)	13
2.4.2 Tầng Giải mã chỉ thị (Instruction Decode - ID)	14
2.4.3 Tầng Thực thi (Execute - EX)	15
2.4.4 Tầng Truy cập bộ nhớ (Memory Access - MEM)	16
2.4.5 Tầng Ghi ngược (Write Back - WB)	16
2.5 Thiết kế Khối điều khiển (Control Unit) đơn nhân	17
2.5.1 Danh sách các tín hiệu điều khiển	17
2.5.2 Bộ tạo hằng số mở rộng (Immediate Generator)	18
2.5.3 Bảng chân trị các tín hiệu điều khiển	18
2.6 Cơ chế xử lý xung đột trong Pipeline	19
2.7 Bộ phân xử Arbiter và giao dịch nguyên tử	19
3 Kết Quả Mô Phỏng Và Thảo Luận	20
3.1 Kết quả kiểm thử đơn nhân	20
3.2 Kết quả kiểm thử tích hợp 8 lõi	20
3.3 Thảo luận về hiệu năng hệ thống	21

4	Hạn Chế Và Hướng Phát Triển	22
4.1	Hạn chế hiện tại của đề tài	22
4.2	Hướng phát triển đề tài	22
	Tài Liệu Tham Khảo	23

Chương 1: Giới Thiệu Đề Tài

1.1 Bối cảnh nghiên cứu

Trong sự phát triển của kiến trúc máy tính hiện đại, việc cải tiến hiệu năng bộ vi xử lý bằng cách tăng tần số xung nhịp đã đạt tới giới hạn do hiện tượng quá nhiệt và tiêu hao năng lượng cực lớn (Power Wall, Thermal Wall). Giải pháp thay thế tất yếu là chuyển dịch sang kiến trúc tính toán song song, tích hợp nhiều lõi xử lý hoạt động độc lập (Multi-core Processing) trên cùng một chip. Đề tài này mô phỏng một hệ thống đa lõi nhằm giải quyết bài toán giao tiếp, tranh chấp bộ nhớ và đồng bộ giữa các luồng xử lý phân cứng.

1.2 Chủ đề của dự án

Dự án tập trung nghiên cứu, thiết kế phần cứng ở mức RTL sử dụng ngôn ngữ mô tả phần cứng Verilog HDL để mô phỏng bộ xử lý 8 lõi độc lập. Mỗi lõi CPU được xây dựng dựa trên kiến trúc tập lệnh mã nguồn mở RISC-V (RV32I subset) và áp dụng kỹ thuật thiết kế pipeline 5 tầng nhằm tối ưu hóa thông lượng lệnh thực thi. Hệ thống 8 nhân này sử dụng chung một khối bộ nhớ dữ liệu (Shared Data Memory) thông qua một bộ phân xử Bus dùng chung.

1.3 Mục tiêu thiết kế

- **Thiết kế lõi đơn:** Thiết kế lõi xử lý RISC-V có pipeline 5 tầng hoàn chỉnh, xử lý xung đột lệnh một cách tối ưu.
- **Thiết kế trục bus đa lõi:** Xây dựng cơ chế phân xử công bằng, đảm bảo cả 8 nhân CPU đều có thể ghi/đọc bộ nhớ dùng chung mà không xảy ra xung đột hay deadlock.
- **Mô phỏng chức năng:** Sử dụng bộ công cụ nguồn mở Icarus Verilog để chạy mô phỏng và GTKWave để phân tích giản đồ thời gian.

Chương 2: Phương Pháp Luận & Kiến Trúc Hệ Thống

2.1 Phương pháp luận thiết kế hệ thống

Để xây dựng thành công hệ thống vi xử lý đa lõi 8-Core RISC-V hoạt động ổn định và tối ưu, dự án áp dụng phương pháp luận thiết kế phần cứng hệ thống theo các nguyên tắc cốt lõi sau:

2.1.1 Quy trình thiết kế Top-Down kết hợp Bottom-Up

Quy trình xây dựng hệ thống kết hợp hài hòa hai cách tiếp cận:

- **Thiết kế từ trên xuống (Top-Down Design):** Xuất phát từ yêu cầu hệ thống đa nhân chia sẻ bộ nhớ và sử dụng tập lệnh RISC-V. Hệ thống được phân rã thành các khối chức năng độc lập bao gồm: Lõi đơn nhân (Core), Bộ phân xử (Arbiter), Bộ điều khiển Bus (Bus Controller) và Bộ nhớ dữ liệu dùng chung (Shared RAM). Việc xác định rõ giao tiếp (Interface) giữa các khối này là bước tiên quyết trước khi viết mã RTL.
- **Tích hợp từ dưới lên (Bottom-Up Integration):** Hiện thực hóa và kiểm thử kỹ lưỡng từ mức linh kiện cơ bản: Đơn vị số học logic (ALU), Tập thanh ghi (Register File), Bộ điều khiển (Control Unit), Logic cập nhật PC. Sau đó tích hợp thành Lõi xử lý đơn (Core) và trực liên kết đa nhân (Interconnect). Cuối cùng là tích hợp toàn hệ thống 8-Core và kiểm chứng bằng testbench hệ thống phức tạp.

2.1.2 Mô hình hóa phần cứng ở mức RTL (RTL Modeling)

Toàn bộ hệ thống được mô tả bằng ngôn ngữ mô tả phần cứng **Verilog HDL** (chuẩn IEEE 1364-2001). Phương pháp thiết kế RTL tập trung vào:

- **Tách biệt mạch tuần tự và mạch tổ hợp:** Sử dụng sườn lên xung nhịp (`posedge clk`) để chốt dữ liệu vào các thanh ghi pipeline và bộ đếm chương trình PC. Sử dụng khối `always @(*)` và `assign` cho các mạch tổ hợp giải mã lệnh và tính toán ALU.
- **Thiết kế modular hóa:** Mỗi khối chức năng được đóng gói trong một file riêng biệt, giao tiếp qua các cổng dữ liệu (ports) rõ ràng, giúp dễ dàng bảo trì và kiểm thử độc lập.

2.1.3 Quy trình mô phỏng và kiểm chứng dựa trên công cụ mã nguồn mở

Kiểm chứng chức năng (Functional Verification) đóng vai trò sống còn. Phương pháp luận kiểm chứng của dự án dựa trên:

- **Mô phỏng mức RTL (RTL Simulation):** Sử dụng công cụ mã nguồn mở **Icarus Verilog** để biên dịch mã nguồn và chạy các testbench. Các kịch bản kiểm thử được thiết kế để bao

phủ các tình huống xung đột dữ liệu (data hazard), xung đột điều khiển (control hazard) và tranh chấp bus bộ nhớ.

- **Phân tích giản đồ dạng sóng (Waveform Analysis):** Xuất tệp dạng sóng định dạng VCD (Value Change Dump) và sử dụng phần mềm trực quan **GTKWave** để gỡ lỗi (debug) các hành vi phần cứng theo từng chu kỳ xung nhịp.

2.1.4 Mô hình hóa chia sẻ tài nguyên và cơ chế đồng bộ

Đối với hệ thống 8 lõi dùng chung bộ nhớ:

- **Tránh deadlock và starvation:** Áp dụng thuật toán phân xử Round-Robin cho bộ Arbiter nhằm cấp quyền truy cập bus tuần hoàn, công bằng giữa các nhân.
- **Đồng bộ hóa phần cứng:** Hỗ trợ các chỉ thị lệnh nguyên tử (Atomic) từ nhóm mở rộng A-Extension nhằm giải quyết xung đột ghi/đọc đồng thời từ phần mềm đa nhân.

2.2 Kiến trúc tập lệnh RISC-V (ISA) áp dụng trong dự án

Trước khi đi vào thiết kế phần cứng ở mức RTL, việc hiểu rõ kiến trúc tập lệnh (Instruction Set Architecture - ISA) là bắt buộc. ISA đóng vai trò là đặc tả giao tiếp giữa phần mềm và phần cứng. Trong dự án này, hệ thống vi xử lý được xây dựng dựa trên tập chỉ thị **RV32I** kết hợp với một số chỉ thị thuộc nhóm mở rộng nguyên tử **A-Extension** (RV32IA subset).

2.2.1 Tổng quan về tập chỉ thị RV32I

RV32I là tập chỉ thị số nguyên cơ bản dạng 32-bit của RISC-V. Đây là tập chỉ thị bắt buộc và là nền tảng cho mọi bộ xử lý RISC-V. Các đặc trưng kỹ thuật chính bao gồm:

- **Độ rộng địa chỉ và thanh ghi:** 32-bit cho cả thanh ghi đa năng và không gian địa chỉ bộ nhớ.
- **Cơ chế nạp/lưu (Load-Store Architecture):** Chỉ các chỉ thị Load (LW) và Store (SW) mới được phép truy cập bộ nhớ dữ liệu. Các phép toán toán học và logic chỉ thực hiện trực tiếp trên tập thanh ghi.
- **Độ dài lệnh cố định:** Mọi chỉ thị lệnh đều có độ rộng cố định là 32-bit, giúp đơn giản hóa quá trình nạp và giải mã lệnh trong pipeline phần cứng.

2.2.2 Tập thanh ghi đa năng (Register File)

Bộ xử lý tích hợp 32 thanh ghi đa năng kí hiệu từ x0 đến x31. Trong đó, thanh ghi x0 (zero) được nối cứng về mức logic 0 ở mức phần cứng và không thể thay đổi giá trị. Bảng 2.1 dưới đây mô tả chi tiết chức năng và quy ước đặt tên (ABI) của tập thanh ghi:

Thanh ghi	Tên ABI	Mô tả chức năng	Lưu giữ
x0	zero	Luôn bằng 0 (Hằng số phần cứng)	–
x1	ra	Địa chỉ trả về của hàm (Return Address)	Caller
x2	sp	Con trỏ ngăn xếp (Stack Pointer)	Callee
x3	gp	Con trỏ toàn cục (Global Pointer)	–
x4	tp	Con trỏ luồng/tiểu trình (Thread Pointer)	–
x5-x7	t0-t2	Thanh ghi tạm thời (Temporaries)	Caller
x8	s0/fp	Thanh ghi lưu giữ / Con trỏ khung (Frame Pointer)	Callee
x9	s1	Thanh ghi lưu giữ (Saved register)	Callee
x10-x11	a0-a1	Tham số truyền vào / Kết quả trả về của hàm	Caller
x12-x17	a2-a7	Các tham số truyền vào hàm (Arguments)	Caller
x18-x27	s2-s11	Các thanh ghi lưu giữ (Saved registers)	Callee
x28-x31	t3-t6	Các thanh ghi tạm thời (Temporaries)	Caller

Bảng 2.1: Quy ước chức năng tập thanh ghi trong kiến trúc RISC-V

2.2.3 Các kiểu định dạng chỉ thị lệnh cơ bản

Kiến trúc RISC-V phân chia mã lệnh 32-bit thành 6 kiểu định dạng cơ bản để tối ưu bộ giải mã:

- **R-type (Register):** Dành cho các lệnh tính toán số học/logic giữa các thanh ghi nguồn (rs1, rs2) và ghi kết quả vào thanh ghi đích (rd).
- **I-type (Immediate):** Dành cho các lệnh tính toán với hằng số nạp trực tiếp 12-bit (imm) hoặc lệnh Load bộ nhớ.
- **S-type (Store):** Dành cho chỉ thị Store lưu dữ liệu từ thanh ghi nguồn vào bộ nhớ.
- **B-type (Branch):** Dành cho chỉ thị so sánh rẽ nhánh có điều kiện với độ lệch địa chỉ 12-bit.
- **U-type (Upper Immediate):** Dành cho chỉ thị nạp hằng số lớn 20-bit vào nửa cao của thanh ghi (như LUI, AUIPC).
- **J-type (Jump):** Dành cho chỉ thị nhảy không điều kiện (JAL).

2.2.4 Đặc tả định dạng mã hóa và trường bit của các lệnh thực hiện

Bộ xử lý trong dự án này hiện thực một tập con của kiến trúc tập lệnh RV32I cùng nhóm mở rộng nguyên tử A-Extension để điều khiển và đồng bộ tính toán đa lõi. Dưới đây là bảng đặc tả chi tiết mã hóa nhị phân 32-bit cho từng chỉ thị lệnh được triển khai thực tế trong mã nguồn RTL:

1. Nhóm lệnh định dạng R (R-type)

Các lệnh tính toán số học và logic thực thi trực tiếp giữa các thanh ghi nguồn ($rs1$, $rs2$) và lưu kết quả vào thanh ghi đích (rd). Chúng sử dụng chung mã opcode là 0110011 (7'h33). Chi tiết mã hóa các trường bit được trình bày trong Bảng 2.2:

2. Nhóm lệnh định dạng I (I-type)

Các lệnh tính toán với hằng số nạp trực tiếp (imm), lệnh nạp dữ liệu từ bộ nhớ (LW), hoặc lệnh nhảy gián tiếp (JALR) có độ dài hằng số là 12-bit.

- Lệnh tính toán ALU sử dụng opcode 0010011 (7'h13).
- Lệnh đọc bộ nhớ LW sử dụng opcode 0000011 (7'h03).
- Lệnh nhảy gián tiếp JALR sử dụng opcode 1100111 (7'h67).

Chi tiết mã hóa được trình bày trong Bảng 2.3. Đối với hai chỉ thị dịch bit SLLI và SRLI, hằng số dịch ($shamt$) chỉ có độ rộng 5-bit (để dịch tối đa 31 bit), do đó trường $imm[11:5]$ được nối cứng bằng 0000000.

3. Nhóm lệnh định dạng S (S-type)

Định dạng dùng cho lệnh ghi dữ liệu vào bộ nhớ RAM dùng chung. Mã opcode duy nhất của nhóm là 0100011 (7'h23). Trường hằng số dịch chuyển địa chỉ (imm) được phân tách thành hai phần đặt ở hai vị trí khác nhau để tối ưu hóa đường dây giải mã trong CPU. Chi tiết mã hóa lệnh SW được mô tả trong Bảng 2.4:

4. Nhóm lệnh định dạng B (Branch - B-type)

Các lệnh rẽ nhánh có điều kiện so sánh giá trị giữa $rs1$ và $rs2$ để quyết định việc thay đổi địa chỉ PC. Chúng sử dụng chung mã opcode 1100011 (7'h63). Các bit của hằng số nhảy lệch được phân rã phức tạp nhằm đồng nhất với cấu trúc mạch giải mã của định dạng S. Chi tiết cấu trúc được thể hiện qua Bảng 2.5:

5. Nhóm lệnh định dạng U (U-type) và J (J-type)

Các chỉ thị làm việc với hằng số lớn nạp trực tiếp 20-bit vào nửa cao của thanh ghi (LUI, AUIPC) hoặc thực hiện lệnh nhảy không điều kiện tầm xa (JAL). Chi tiết cấu trúc mã hóa được thể hiện qua Bảng 2.6:

6. Nhóm lệnh nguyên tử đồng bộ (A-Extension)

Đặc trưng thiết yếu của hệ thống 8 lõi xử lý chia sẻ RAM dữ liệu là việc đồng bộ hóa đa luồng thông qua các chỉ thị nguyên tử (Atomic). Các chỉ thị này sử dụng chung mã opcode 0101111 (7'h2F) và trường funct3 mặc định là 010 (độ rộng dữ liệu word 32-bit). Lệnh được phân biệt bởi mã trường 5 bit funct5 = instruction[31:27]. Để đơn giản hóa mạch phần cứng, hai bit điều khiển tính nhất quán bộ nhớ là aq (bit 26) và rl (bit 25) được nối cứng về 0. Chi tiết mã hóa của các lệnh nguyên tử được thể hiện qua Bảng 2.7:

2.2.5 Các bảng tổng hợp đặc tả mã hóa chỉ thị lệnh

Dưới đây là toàn bộ các bảng đặc tả mã hóa nhị phân chi tiết cho từng nhóm chỉ thị lệnh được hiện thực hóa trong dự án:

Lệnh	funct7 [31:25]	rs2 [24:20]	rs1 [19:15]	funct3 [14:12]	rd [11:7]	opcode [6:0]	Mô tả hoạt động
ADD	0000000	rs2	rs1	000	rd	0110011	$R[rd] = R[rs1] + R[rs2]$
SUB	0100000	rs2	rs1	000	rd	0110011	$R[rd] = R[rs1] - R[rs2]$
SLL	0000000	rs2	rs1	001	rd	0110011	$R[rd] = R[rs1] \ll R[rs2][4:0]$
SLT	0000000	rs2	rs1	010	rd	0110011	$R[rd] = (R[rs1] < R[rs2]) ? 1 : 0$
XOR	0000000	rs2	rs1	100	rd	0110011	$R[rd] = R[rs1] \oplus R[rs2]$
SRL	0000000	rs2	rs1	101	rd	0110011	$R[rd] = R[rs1] \gg R[rs2][4:0]$
OR	0000000	rs2	rs1	110	rd	0110011	$R[rd] = R[rs1] \mid R[rs2]$
AND	0000000	rs2	rs1	111	rd	0110011	$R[rd] = R[rs1] \& R[rs2]$

Bảng 2.2: Mã hóa chi tiết các lệnh định dạng R-type

Lệnh	imm[11:0] [31:20]	rs1 [19:15]	funct3 [14:12]	rd [11:7]	opcode [6:0]	Mô tả hoạt động
ADDI	imm[11:0]	rs1	000	rd	0010011	$R[rd] = R[rs1] + imm$
SLTI	imm[11:0]	rs1	010	rd	0010011	$R[rd] = (R[rs1] < imm) ? 1 : 0$
XORI	imm[11:0]	rs1	100	rd	0010011	$R[rd] = R[rs1] \sim imm$
ORI	imm[11:0]	rs1	110	rd	0010011	$R[rd] = R[rs1] imm$
ANDI	imm[11:0]	rs1	111	rd	0010011	$R[rd] = R[rs1] \& imm$
SLLI	0000000 shamt[4:0]	rs1	001	rd	0010011	$R[rd] = R[rs1] \ll shamt$
SRLI	0000000 shamt[4:0]	rs1	101	rd	0010011	$R[rd] = R[rs1] \gg shamt$
LW	imm[11:0]	rs1	010	rd	0000011	$R[rd] = M[R[rs1] + imm]$
JALR	imm[11:0]	rs1	000	rd	1100111	$R[rd] = PC + 4; PC = (R[rs1] + imm) \& \sim 1$

Bảng 2.3: Mã hóa chi tiết các lệnh định dạng I-type

Lệnh	imm[11:5] [31:25]	rs2 [24:20]	rs1 [19:15]	funct3 [14:12]	imm[4:0] [11:7]	opcode [6:0]	Mô tả hoạt động
SW	imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	$M[R[rs1] + imm] = R[rs2]$

Bảng 2.4: Mã hóa chi tiết lệnh định dạng S-type

Lệnh	imm[12 10:5] [31:25]	rs2 [24:20]	rs1 [19:15]	funct3 [14:12]	imm[4:1 11] [11:7]	opcode [6:0]	Điều kiện rẽ nhánh
BEQ	imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	if (R[rs1] == R[rs2]) PC = PC + imm
BNE	imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	if (R[rs1] != R[rs2]) PC = PC + imm
BLT	imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	if (R[rs1] < R[rs2]) PC = PC + imm
BGE	imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	if (R[rs1] >= R[rs2]) PC = PC + imm

Bảng 2.5: Mã hóa chi tiết các lệnh định dạng B-type

Lệnh	imm [31:12]	rd [11:7]	opcode [6:0]	Mô tả hoạt động
LUI	imm[31:12]	rd	0110111	R[rd] = imm « 12
AUIPC	imm[31:12]	rd	0010111	R[rd] = PC + (imm « 12)
JAL	imm[20 10:1 11 19:12]	rd	1101111	R[rd] = PC + 4; PC = PC + imm

Bảng 2.6: Mã hóa chi tiết các lệnh định dạng U-type và J-type

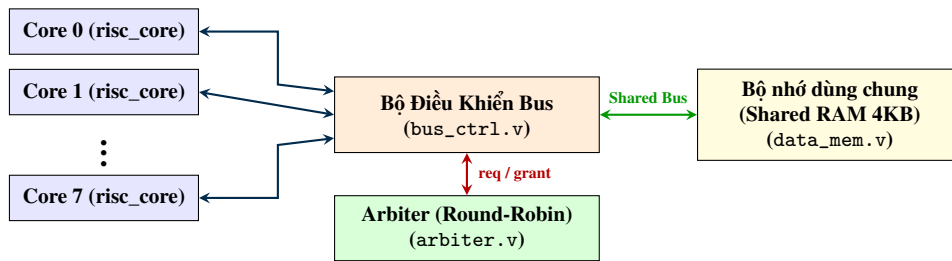
Lệnh	funct5 [31:27]	aq [26]	rl [25]	rs2 [24:20]	rs1 [19:15]	funct3 [14:12]	rd [11:7]	opcode [6:0]	Mô tả hoạt động
LR.W	00010	0	0	00000	rs1	010	rd	0101111	$R[rd] = M[R[rs1]]$ (đặt khóa)
SC.W	00011	0	0	rs2	rs1	010	rd	0101111	$R[rd] =$ $SC(M[R[rs1]]) =$ $R[rs2]$
AMOSWAP.W	00001	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] = R[rs2]$
AMOADD.W	00000	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] = R[rd]$ $+ R[rs2]$
AMOAND.W	01100	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] = R[rd]$ $\& R[rs2]$
AMOOR.W	01010	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] = R[rd]$ $ R[rs2]$
AMOXOR.W	00100	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] = R[rd]$ $\sim R[rs2]$
AMOMIN.W	10000	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] =$ $\min(R[rd], R[rs2])$
AMOMAX.W	10100	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] =$ $\max(R[rd], R[rs2])$
AMOMINU.W	11000	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] =$ $\min_u(R[rd],$ $R[rs2])$
AMOMAXU.W	11100	0	0	rs2	rs1	010	rd	0101111	$R[rd] = M[R[rs1]];$ $M[R[rs1]] =$ $\max_u(R[rd],$ $R[rs2])$

Bảng 2.7: Mã hóa chi tiết các lệnh nguyên tử thuộc nhóm A-Extension

2.3 Sơ đồ cấu trúc tổng thể hệ thống 8-Core

Kiến trúc hệ thống bao gồm 8 lõi CPU chạy song song độc lập. Các CPU có bộ nhớ lệnh (Instruction memory) riêng nhằm triệt tiêu xung đột truy cập lệnh, nhưng chia sẻ một khối RAM dữ liệu dùng chung (Shared Data Memory). Sự giao tiếp này được quản lý thông qua hai khối chính là:

1. **Bộ phân xử Bus (Arbiter - arbiter.v):** Phân xử quyền truy cập dựa trên thuật toán Round-Robin.
2. **Bộ điều khiển Bus (Bus Controller - bus_ctrl.v):** Định tuyến các bus địa chỉ, dữ liệu đầu vào và đầu ra đến RAM dữ liệu tương ứng của nhân được cấp quyền.



Hình 2.1: Sơ đồ cấu trúc kết nối tổng thể hệ thống 8-Core

2.4 Thiết kế đường truyền dữ liệu (Datapath) đơn nhân

Đường truyền dữ liệu (Datapath) của lõi vi xử lý đơn nhân (risc_core.v) được thiết kế theo kiến trúc Pipeline 5 tầng tuần tự. Dưới đây là mô tả chi tiết dòng chảy dữ liệu, các bộ chọn (Multiplexer) và các tín hiệu điều khiển tương ứng tại từng giai đoạn xử lý:



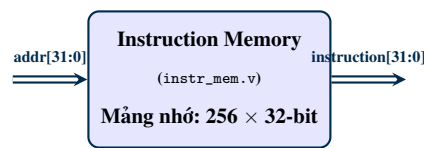
Hình 2.2: Phân bố các khối chức năng trên 5 tầng Pipeline đơn nhân

2.4.1 Tầng Nạp chỉ thị (Instruction Fetch - IF)

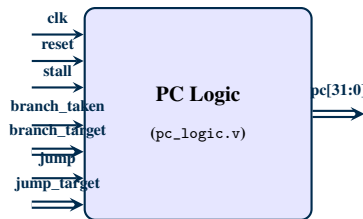
Nhiệm vụ chính của tầng IF là lấy mã lệnh từ bộ nhớ lệnh riêng (instr_mem.v) dựa trên địa chỉ của bộ đếm chương trình PC.

- **Cơ chế cập nhật PC:** Tại mỗi chu kỳ xung nhịp, PC được cập nhật dựa trên mức độ ưu tiên giảm dần:

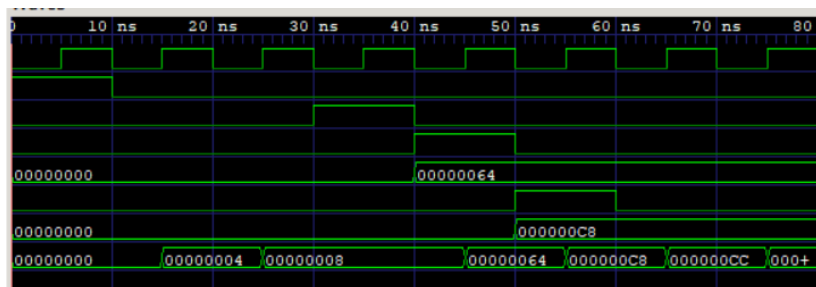
1. *Reset*: Đưa địa chỉ PC về 32'd0.
 2. *Stall*: Khi phát hiện xung đột nạp dữ liệu ($\text{load_use_stall} = 1$) hoặc phải chờ bus bộ nhớ ($\text{mem_wait} = 1$), PC giữ nguyên giá trị cũ để hoãn pipeline.
 3. *Branch/Jump Taken*: Khi xảy ra rẽ nhánh hoặc nhảy được xác định tại tầng EX ($\text{redirect_taken} = 1$), PC sẽ nạp địa chỉ đích mới (redirect_target).
 4. *Mặc định*: Tăng tiền địa chỉ đều đặn $\text{PC} \leq \text{PC} + 32'd4$.
- **Thanh ghi đệm IF/ID**: Cuối chu kỳ IF, mã lệnh lấy ra (instr_from_mem) và địa chỉ PC tương ứng được lưu lại vào các thanh ghi đệm if_id_instr và if_id_pc để đẩy sang tầng tiếp theo.



Hình 2.3: Sơ đồ khối bộ nhớ lệnh (instr_mem.v)



Hình 2.4: Sơ đồ khối bộ điều phối PC (pc_logic.v)

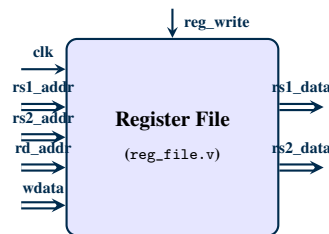


Hình 2.5: Giải đồ xung mô phỏng của bộ PC Logic (pc_logic_wave.png)

2.4.2 Tầng Giải mã chỉ thị (Instruction Decode - ID)

Tầng ID thực hiện giải mã lệnh để trích xuất các tín hiệu điều khiển và chuẩn bị các toán hạng số học.

- **Giải mã trung tâm (control_unit.v):** Trích xuất các trường từ lệnh `if_id_instr` để sinh ra các tín hiệu điều khiển đường truyền dữ liệu (như ghi thanh ghi `cu_reg_write`, chọn nguồn ALU `cu_alu_src`, chọn ghi ngược `cu_mem_to_reg`, rẽ nhánh `cu_branch`, nhảy `cu_jump`, các chỉ thị nguyên tử `cu_mem_lr`, `cu_mem_sc`, `cu_mem_amo`).
- **Bộ tạo hằng số (Immediate Generator):** Giải mã và căn lề dấu (Sign-extend) tạo hằng số mở rộng 32-bit `cu_imm` phù hợp với các định dạng lệnh I, S, B, U, J.
- **Tập thanh ghi (reg_file.v):** Đọc bất đồng bộ dữ liệu từ hai địa chỉ thanh ghi nguồn `rs1_addr` và `rs2_addr` để tạo ra hai toán hạng `rs1_data` và `rs2_data`.
- **Thanh ghi đệm ID/EX:** Tất cả các tín hiệu điều khiển giải mã cùng dữ liệu toán hạng được chốt lại vào các thanh ghi đệm pipeline `id_ex_*` ở sườn lên clock tiếp theo.



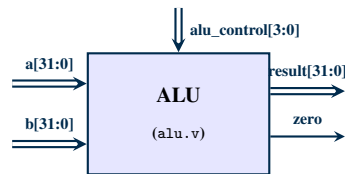
Hình 2.6: Sơ đồ khối tập thanh ghi (reg_file.v)

2.4.3 Tầng Thực thi (Execute - EX)

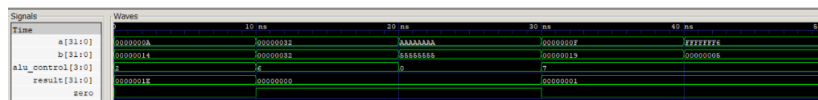
Tầng EX là trung tâm xử lý số học và điều khiển luồng chương trình.

- **Mạch Forwarding (Bơm tắt kết quả):** Để loại bỏ độ trễ do xung đột dữ liệu, hệ thống tích hợp bộ chọn (MUX) cho hai toán hạng đầu vào ALU. Bộ chọn sẽ ưu tiên bơm kết quả trực tiếp từ tầng trước thay vì đọc từ tập thanh ghi:
 - Lấy kết quả từ tầng EX/MEM (`ex_mem_result`) nếu lệnh trước đó ghi thanh ghi và trùng địa chỉ nguồn thanh ghi đích.
 - Lấy kết quả ghi ngược từ tầng MEM/WB (`mem_wb_write_data`) nếu lệnh trước nữa trùng địa chỉ nguồn thanh ghi đích.
 - Sử dụng toán hạng đọc trực tiếp từ Register File nếu không có xung đột xảy ra.
- **Bộ chọn nguồn vào ALU (ALU Source MUX):** Toán hạng thứ nhất của ALU (`alu_input_a`) luôn là dữ liệu nguồn 1 sau khi qua mạch forwarding (`fwd_rs1_data`). Toán hạng thứ hai (`alu_input_b`) được chọn qua bộ MUX 2-sang-1: chọn hằng số `id_ex_imm` nếu tín hiệu `id_ex_alu_src = 1`, hoặc chọn dữ liệu nguồn 2 sau forwarding (`fwd_rs2_data`) nếu `id_ex_alu_src = 0`.

- **Khối ALU (alu.v):** Thực thi tính toán tạo ra kết quả alu_result và cờ bằng không alu_zero.
- **Khối xử lý Rẽ nhánh & Nhảy:** So sánh hai toán hạng fwd_rs1_data và fwd_rs2_data (hỗ trợ BEQ, BNE, BLT, BGE). Nếu điều kiện rẽ nhánh thỏa mãn hoặc là lệnh nhảy, tín hiệu chuyển hướng redirect_taken sẽ được kích hoạt, địa chỉ nhảy redirect_target được tính toán để đưa ngược về tầng IF. Đồng thời, một lệnh xóa (Flush) được phát đi để làm trống các thanh ghi đệm tầng nạp chỉ thị sai.



Hình 2.7: Sơ đồ khối đơn vị số học & logic ALU (alu.v)



Hình 2.8: Giải đồ xung mô phỏng của bộ tính toán ALU (alu_wave.png)

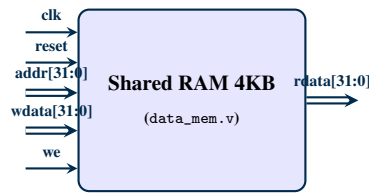
2.4.4 Tầng Truy cập bộ nhớ (Memory Access - MEM)

Thực hiện các giao dịch đọc và ghi dữ liệu với bộ nhớ dùng chung.

- **Định tuyến yêu cầu:** Khi gặp lệnh load/store hoặc atomic, tầng MEM kích hoạt tín hiệu mem_req = 1, đưa địa chỉ (ex_mem_result) và dữ liệu ghi (ex_mem_store_data) ra bus ngoài thông qua bộ liên kết Interconnect.
- **Đồng bộ hóa đợi Bus (Memory Stall):** Do RAM dữ liệu là tài nguyên dùng chung, lõi CPU phải đợi tín hiệu phản hồi mem_ready = 1 từ Bus Controller. Trong quá trình chờ đợi này, tín hiệu hoãn mem_wait được kích hoạt làm đóng băng toàn bộ các tầng pipeline phía sau (IF, ID, EX) để dữ liệu không bị ghi đè chéo.
- **Chốt dữ liệu đọc:** Dữ liệu đọc từ RAM (mem_rdata) hoặc kết quả ghi điều kiện (mem_sc_result) được chốt vào thanh ghi đệm mem_wb_* khi bus phản hồi hoàn tất giao dịch.

2.4.5 Tầng Ghi ngược (Write Back - WB)

Giai đoạn cuối cùng đưa dữ liệu thực thi trở về tập thanh ghi.



Hình 2.9: Sơ đồ khối bộ nhớ chia sẻ dữ liệu (`data_mem.v`)

- **Bộ chọn ghi ngược (Write Back MUX):** Lựa chọn dữ liệu ghi cuối cùng (`mem_wb_write_data`) từ các nguồn:
 - Dữ liệu đọc được từ RAM (`mem_rdata`) đối với lệnh Load (LW, LR.W).
 - Kết quả kiểm tra ghi điều kiện (`mem_sc_result`) đối với lệnh SC.W.
 - Kết quả tính toán ALU (`ex_mem_result`) đối với các lệnh tính toán số học hoặc rẽ nhánh, nhảy.
- **Cập nhật tập thanh ghi:** Khẳng định tín hiệu `rf_write_en = 1` để ghi đồng bộ dữ liệu vào tập thanh ghi tại sườn lên clock tiếp theo tại địa chỉ thanh ghi đích `mem_wb_rd_addr` (loại trừ thanh ghi cứng x0).

2.5 Thiết kế Khối điều khiển (Control Unit) đơn nhân

Khối điều khiển (`control_unit.v`) chịu trách nhiệm giải mã mã lệnh 32-bit nhận được từ bộ nhớ chương trình, sinh ra các tín hiệu điều khiển tương ứng để cấu hình đường truyền dữ liệu (Datapath), ALU, khối giao tiếp bộ nhớ và bộ tạo hằng số mở rộng.

2.5.1 Danh sách các tín hiệu điều khiển

Các tín hiệu điều khiển chính được tạo ra bao gồm:

- `reg_write`: Cho phép ghi kết quả vào tập thanh ghi đa năng.
- `alu_src`: Lựa chọn nguồn vào thứ hai (B) cho ALU (0: đọc từ thanh ghi nguồn `rs2_data`, 1: sử dụng hằng số mở rộng `imm`).
- `alu_control[3:0]`: Xác định mã phép toán cho bộ ALU thực thi (ví dụ: cộng, trừ, dịch bit, toán tử logic).
- `mem_read / mem_write`: Tín hiệu yêu cầu đọc/ghi dữ liệu tương ứng đối với bộ nhớ RAM chia sẻ.
- `mem_to_reg`: Xác định nguồn ghi ngược trở lại thanh ghi đích `rd` (0: kết quả từ ALU, 1: dữ liệu đọc lên từ bộ nhớ dữ liệu).

- **branch / branch_type[2:0]**: Tín hiệu kích hoạt rẽ nhánh có điều kiện và loại điều kiện rẽ nhánh (BEQ, BNE, BLT, BGE).
- **jump / jalr**: Tín hiệu điều khiển nhảy không điều kiện (JAL) hoặc nhảy gián tiếp qua thanh ghi (JALR).
- **lui / auipc**: Các tín hiệu đặc biệt cho các chỉ thị nạp hằng số lớn vào nửa cao thanh ghi.
- **mem_lr / mem_sc / mem_amo / amo_op[3:0]**: Các tín hiệu kích hoạt và xác định toán tử nguyên tử tương ứng trong tập lệnh mở rộng nhóm A-Extension để thực hiện đồng bộ đa nhân.

2.5.2 Bộ tạo hằng số mở rộng (Immediate Generator)

Bộ tạo hằng số tích hợp bên trong khối điều khiển tự động trích xuất các trường bit của lệnh và thực hiện mở rộng dấu (sign-extension) thành số 32-bit hoàn chỉnh tùy theo định dạng của từng nhóm lệnh:

- **I-type** (ADDI, LW, JALR): Trích xuất `instruction[31:20]`.
- **S-type** (SW): Trích xuất và ghép nối `{instruction[31:25], instruction[11:7]}`.
- **B-type** (BEQ...): Trích xuất, sắp xếp lại các trường bit và chèn thêm bit 0 ở cuối để tính địa chỉ rẽ nhánh nửa từ.
- **U-type** (LUI, AUIPC): Trích xuất `instruction[31:12]` và nối thêm 12 bit 0 vào nửa thấp.
- **J-type** (JAL): Trích xuất và sắp xếp lại các bit của khoảng dịch địa chỉ 20-bit, sau đó dịch trái 1 bit (chèn bit 0 thấp).

2.5.3 Bảng chân trị các tín hiệu điều khiển

Giá trị chi tiết của các tín hiệu điều khiển tương ứng với từng opcode được thiết lập theo Bảng [2.8](#):

Nhóm lệnh	Opcode [6:0]	Reg Write	ALU Src	ALU Control	Mem Read	Mem Write	Mem toReg	Branch	Jump	Atomic
R-type	0110011	1	0	Theo funct	0	0	0	0	0	0
I-type	0010011	1	1	Theo funct	0	0	0	0	0	0
Load (LW)	0000011	1	1	ADD	1	0	1	0	0	0
Store (SW)	0100011	0	1	ADD	0	1	X	0	0	0
Branch	1100011	0	0	SUB	0	0	X	1	0	0
JAL	1101111	1	X	ADD	0	0	0	0	1	0
JALR	1100111	1	1	ADD	0	0	0	0	1	0
LUI	0110111	1	X	X	0	0	0	0	0	0
AUIPC	0010111	1	X	X	0	0	0	0	0	0
LR.W	0101111	1	1	ADD	0	0	1	0	0	1
SC.W	0101111	1	1	ADD	0	0	0	0	0	1
AMOs	0101111	1	1	ADD	0	0	0	0	0	1

Bảng 2.8: Bảng chân trị giải mã tín hiệu điều khiển của Control Unit

2.6 Cơ chế xử lý xung đột trong Pipeline

- **Data Hazard:** Giải quyết bằng cách chèn các mạch **Forwarding** (bơm tất kết quả từ tầng EX/MEM và MEM/WB trực tiếp về đầu vào ALU tầng EX). Đối với xung đột **Load-use hazard**, bộ xử lý chèn 1 chu kỳ **Stall** (hoãn PC và xóa tín hiệu tầng ID/EX).
- **Control Hazard:** Khi phát hiện branch/jump đi sai hướng, toàn bộ các lệnh đang được nạp dở ở tầng IF/ID và ID/EX sẽ bị xóa (**Flush**) bằng cách đặt cờ hiệu valid của tầng đệm về 0, chuyển lệnh thành chỉ thị rỗng NOP.

2.7 Bộ phân xử Arbiter và giao dịch nguyên tử

Bộ phân xử (arbiter.v) sử dụng cơ chế xoay vòng Round-Robin để đảm bảo tính công bằng. Mỗi nhân khi yêu cầu và được cấp bus (grant = 1) chỉ được giữ bus trong đúng 1 chu kỳ xung nhịp. Bên cạnh đó, bộ điều khiển bus hỗ trợ các tín hiệu điều khiển của lệnh nguyên tử (LR.W / SC.W) giúp phần mềm đa nhân có thể thực hiện khóa đồng bộ tránh tình trạng xung đột cập nhật dữ liệu.

Chương 3: Kết Quả Mô Phỏng Và Thảo Luận

3.1 Kết quả kiểm thử đơn nhân

Kịch bản mô phỏng chạy trên testbench đơn nhân (core_tb.v) cho kết quả hoàn tất chính xác với kết quả:

- retired = 67: Đã hoàn thành và commit 67 chỉ thị lệnh qua pipeline.
- load_use_stall = 1: Phát hiện và xử lý chính xác 1 lần xung đột load-use.
- flush = 62: Thực hiện xóa chính xác 62 chỉ thị lệnh rẽ nhánh không hợp lệ.
- SUMMARY: 7 PASSED, 0 FAILED.

3.2 Kết quả kiểm thử tích hợp 8 lõi

Hệ thống 8 lõi chạy kịch bản kiểm thử ghi và đọc chéo bộ nhớ dùng chung. Các core thực thi ghi giá trị '42' vào 'mem[0]' và ghi giá trị hoàn tất '99' vào 'mem[1]'.

Kết quả thu được từ màn hình console ghi nhận:

```
BAT DAU MO PHONG HE THONG 8 LOI PIPELINE
Kiem tra mem[0] (Ky vong: 42) = 42
[PASS] Shared memory data dung.
Kiem tra mem[1] addr 4 (Ky vong: 99) = 99
[PASS] 8 core hoan thanh chuong trinh khong deadlock.
```

PIPELINE / BUS EVIDENCE

```
Core 0: grant=3 retired=163 mem_stall=13 load_use_stall=0 flush=159
Core 1: grant=3 retired=163 mem_stall=14 load_use_stall=0 flush=159
Core 2: grant=3 retired=163 mem_stall=15 load_use_stall=0 flush=158
Core 3: grant=3 retired=162 mem_stall=16 load_use_stall=0 flush=158
Core 4: grant=3 retired=162 mem_stall=17 load_use_stall=0 flush=157
Core 5: grant=3 retired=162 mem_stall=18 load_use_stall=0 flush=157
Core 6: grant=4 retired=161 mem_stall=19 load_use_stall=0 flush=157
Core 7: grant=4 retired=161 mem_stall=20 load_use_stall=0 flush=157
```

TONG KET: 10 PASSED, 0 FAILED

3.3 Thảo luận về hiệu năng hệ thống

Từ số liệu thống kê thu được:

1. **Tính đúng đắn:** Cả 8 core đều thực thi thành công chương trình mà không xảy ra lỗi dữ liệu hay deadlock.
2. **Đặc trưng Bus trễ:** Số chu kỳ chờ đợi (`mem_stall`) tăng dần từ Core 0 (13 chu kỳ) đến Core 7 (20 chu kỳ). Điều này phản ánh chính xác độ trễ do tranh chấp bus vật lý khi nhiều core cùng truy cập RAM dùng chung một thời điểm, chứng minh tính thực tế của mô hình mô phỏng.

Chương 4: Hạn Chế Và Hướng Phát Triển

4.1 Hạn chế hiện tại của đề tài

- **Hiệu năng kết nối:** Hệ thống chưa trang bị bộ nhớ đệm Cache riêng cho từng lõi CPU dẫn tới tần suất tranh chấp Bus rất cao, gây nghẽn băng thông nghiêm trọng khi chạy các chương trình phức tạp.
- **Tính đầy đủ của tập lệnh:** Tập lệnh chưa hỗ trợ đầy đủ bộ xử lý RISC-V theo chuẩn công nghiệp (chưa có phép nhân chia phân cứng, CSR, hay ngắt ngoại lệ).

4.2 Hướng phát triển đề tài

1. Thiết kế bộ nhớ đệm L1 Cache riêng cho từng core.
2. Hiện thực hóa bộ điều khiển tính nhất quán bộ nhớ đệm (Cache Coherence Controller) theo giao thức MESI hoặc Snooping.
3. Nâng cấp lõi CPU lên kiến trúc RV32IM hoàn chỉnh.

Tài liệu tham khảo

- [1] David A. Patterson và John L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, Morgan Kaufmann, 2017.
- [2] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Version 20191213.
- [3] Steve Harris và Sarah Harris, *Digital Design and Computer Architecture: ARM Edition*, Morgan Kaufmann, 2015.